TPIV Testing for Microfrontends

Motivation

Technical Post Implementation Verification (TPIV) is done for a microfrontend (MFE) with the aim of confirming whether calls that it makes through its experience API are functional and working as expected. These tests can be performed in either Production or lower-level environments. It is recommended that the testing is carried out in lower-level environments for verification purposes, before being carried out in Prod. This document lays out some steps for automating this procedure through tools such as Postman and Github Copilot/Amp Code.

Useful Resources

In addition to this document, the following pages also contain detailed information on motivation as well as setup for your tests. Please read them in conjunction with this document:

- MESH API TPIV in Prod
- MESH API Integration Testing Guidelines

Output

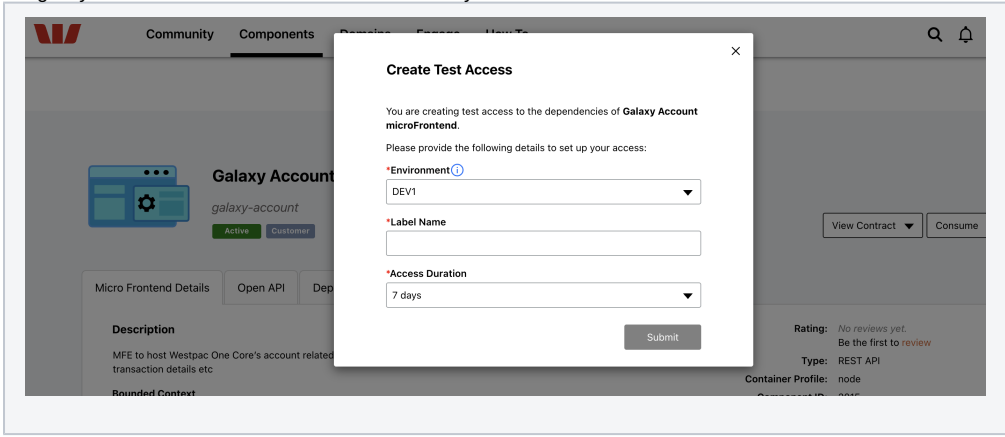
There should only be **TWO** files that are needed for this process:

- ***integration_tests_postman_collection.json*** - the Postman collection that contains all of the required TPIV tests
- ***integration_tests_postman_collection_data.json*** - supplementary data file containing variables that are referenced in integration_tests_postman_collection.json (eg. BSB, organisation IDs, request parameters, etc.)

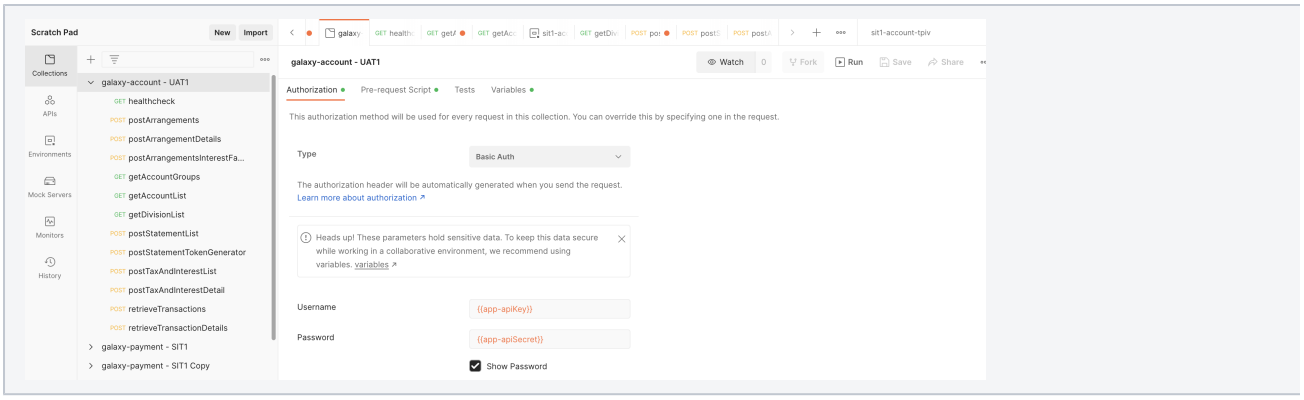
These files should be stored in **api/api-tests/{env_name}**, where `env_name` is the environment for which you are writing these tests (eg. sit1, sit1-w1c1)

Steps

1. Ensure you have Postman installed on your device. You can install an older version here, as this will not require you to create an account in order to create Collections
2. Request test access for the MFE that you would like to test. You can do that by visiting the MESH Connect page for it. For example, if you wanted to request test access for the Account MFE, you would visit this link. You can then click on 'Consume' 'Create Test Access', and fill out the information per your testing needs
3. Click on your Profile Test Access, navigate to your secret that you have just created and download the Postman Environment for this secret. This will give you access to the correct environment with your secrets loaded in as variables



4. Once your Postman environment is set up, create a new Postman collection. Your requests for this MFE should be stored in this Postman collection
5. Ensure that you are using your test access credentials at the collection level through Basic authentication and that they are inherited by each request in the collection, as shown in this image.



6. If the MFE expects an `x-authorization` header, create a dummy JWT for the request. This can be done in two ways:
   a. Manually through https://www.jwt.io/. `x-authorization` expects a JWT, so ensure you are using a Bearer token and not Basic authentication for this header
   b. Automated through the collection's pre-request script, using the following script:

Dynamic JWT generator

```
// Dynamic JWT generation — generates unsigned JWTs at runtime
// so no tokens need to be stored in the data file.
(function () {
    var requestPath = pm.request.url.getPath();

    // Skip healthcheck — no JWT needed
    if (requestPath.indexOf("healthcheck") !== -1) { return; }

    // Derive the resource_uri from the request path
    // The tokenValidationMiddleware checks that the resource_uri path
    // matches the request path (after stripping /customer/ or /staff/ prefix)
    var basePath = "/exp/galaxy/account/v2";
    var idx = requestPath.indexOf(basePath);
    var endpointSegment = (idx !== -1) ? requestPath.substring(idx) : requestPath;
    var resourceUri = "https://a.com/exp/customer/galaxy/account/v2" + endpointSegment.replace
(basePath, "");

    // Build JWT payload
    var payload = {
        sub: "abc",
        iamID: "ca4dd240-30fe-4172-bc95-93fa82fae487",
        resource_uri: resourceUri
    };

    // Base64url encode helper
    function base64urlEncode(str) {
        return btoa(str)
            .replace(/\+/g, "-")
            .replace(/\//g, "_")
            .replace(/=+$/g, "");
    }

    var header = base64urlEncode(JSON.stringify({ alg: "none", typ: "JWT" }));
    var body = base64urlEncode(JSON.stringify(payload));
    var token = "Bearer " + header + "." + body + ".";

    pm.variables.set("generatedBearerToken", token);
})();
```

7. Ensure all required headers have their fields correctly assigned with the expected values, as per the Swagger documentation
8. Test the MFE's EXP API endpoints. For **GET** requests, test whether they can return a successful (eg. 200) response **as well as** a bad request (400) response. For **non-GET** requests (eg. POST, PATCH, DELETE), test for whether they can return a **400 response**. Write the tests in the 'Tests' section of each request. Refer to MESH API Integration Testing Guidelines for more information on what constitutes a good test.
9. Once all endpoints have been tested adequately and have passing tests, export the collection as **integration_tests_postman_collection.json** to the `api/api-tests/{env_name}` folder in your MFE. If you have any supplementary data, export it as **integration_tests_postman_collection_data.json**. This can just be a simple JSON file containing any variables you have referenced in the collection. You **do not** need to have variables for `hostname` or your test access credentials. Let those variables exist as the MESH pipeline will automatically populate these during deployment. **Ensure that no JWTs are exposed in either of the files**.

10. Build the MFE and deploy it to the desired environment. In order to run the TPIV tests, the build must be deployed to one of the following environments:
    a. DEV1 - https://bamboo-wdp.srv.westpac.com.au/browse/WDPTESTDEV1-APITESTDEV1
    b. SIT1 - https://bamboo-wdp.srv.westpac.com.au/browse/WDPTESTSIT1-APITESTSIT1
    c. UAT1 - https://bamboo-wdp.srv.westpac.com.au/browse/WDPTESTUAT1-APITESTUAT1
    d. SVP1 - https://bamboo-wdp.srv.westpac.com.au/browse/WDPTESTSVP1-APITESTSVP1
11. Verify whether the tests have passed.

Debugging

To diagnose issues with why certain tests may be failing, check the `x-appCorrelationId` in Splunk for more information. This can either be done through the web interface or the Splunk MCP.